PBFT

Byzantine Fault Tolerance Consensus Algorithm Simulation

Our team：

Cai Huanghao　　Wang Ying

Zeng Xianrong　　Yu Xiaochen

Distributing data across multiple nodes requires a reliable consensus algorithm.

The Problem: Malicious nodes, message delays, and network failures can all prevent agreement.

Solution: PBFT (Practical Byzantine Fault Tolerance) offers a solution to these challenges.

GOAL OF PROJECT

Construct a PBFT simulator to visualize and understand the consensus process.

The simulator will allow users to customize parameters, configure network conditions, and observe how PBFT works.

Visualize the network topology, monitor node interactions, and analyze consensus performance.

Technology stack

Python 3.6+
with tkinter, random,
collections.defaultdict

ABOUT PBFT

The Byzantine Generals Problem: Imagine a group of generals who must agree on a battle plan.
Some generals (nodes) may be faulty or lie, making agreement impossible.

A Fault-Tolerant Consensus: PBFT solves this challenge by ensuring that honest generals can still reach an agreement.

Core Threshold

safety threshold $f \leq \lfloor \frac{N-1}{3} \rfloor$

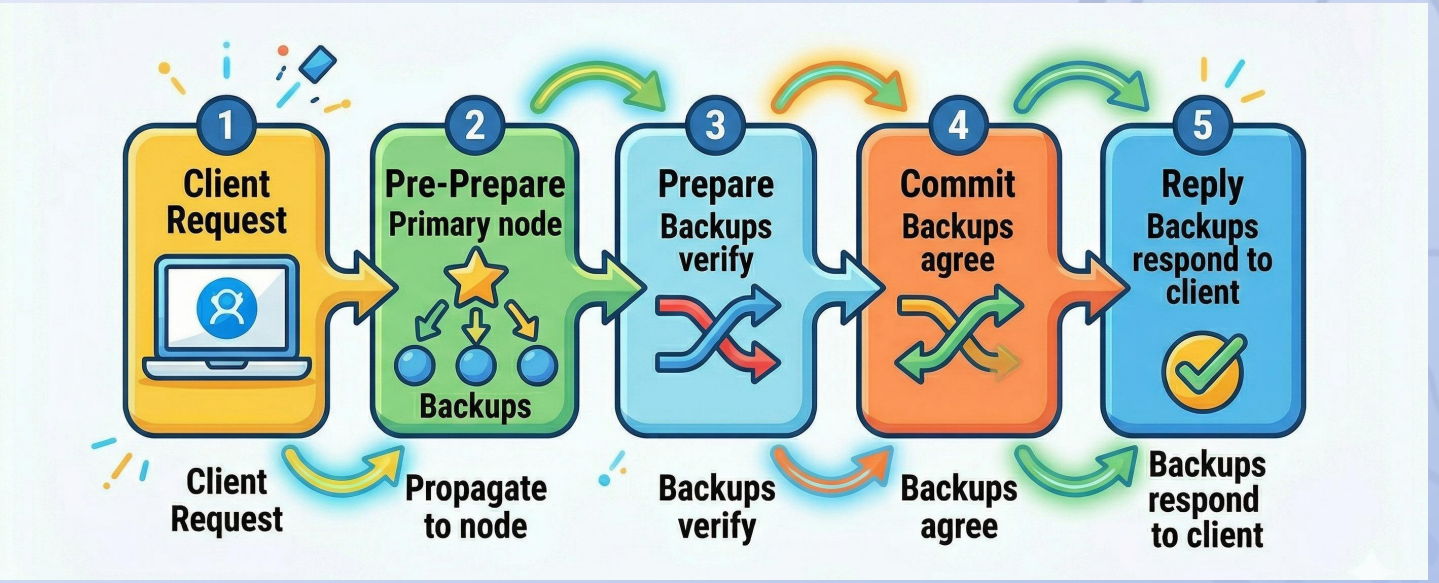
the maximum number of malicious nodes the system can tolerate while maintaining consistency

consensus threshold $T = \lfloor \frac{2N}{3} \rfloor + 1$

the minimum number of identical valid messages required to transition between protocol phases

The Execution Flow

- **Request**: Client sends a request to the Primary node.
- **Pre-Prepare**: Primary sends a pre-prepare message to all Backup nodes.
- **Prepare:** Backup nodes send prepare messages to each other.
- **Commit:** Backup nodes send commit messages to each other.
- **Reply**: Primary and Backup nodes send reply messages to the client.



Realization process

Node Creation
Generate a specified number of nodes, randomly select some as malicious nodes, and fix the primary node as an honest node.

Pre-prepare Phase
The primary node broadcasts an initial proposal; each node receives and processes it (malicious nodes may tamper with it).

Prepare Phase
Honest nodes broadcast prepare messages. Each node counts messages to determine if the threshold is met and verifies proposal validity.

Commit Phase
Nodes that confirm the proposal broadcast commit messages. Each node counts messages to determine if the threshold is met and finalizes the consensus result.

Result Verification
Count final decisions of all honest nodes, verify consistent agreement, and validate the core conclusions of PBFT.

Node Class Attributes:

Attribute	Description
node_id	Unique node identifier
is_malicious	Malicious node flag (True=malicious, False=honest)
preprepare_val	Stores received pre-prepare message (correct value)
prepare_cnt	Counts received prepare messages (dict: proposal=count)
commit_cnt	Counts received commit messages (dict: proposal=count)
prepared	Whether prepared state is reached (True if prepare threshold met)
committed	Whether committed state is reached (True if commit threshold met)
result	Final consensus result (None if uncommitted)

Node Class Attributes:

```python
# Node class: Simulate a single node in PBFT system
class Node:
    def __init__(self, node_id, is_malicious=False):
        self.node_id = node_id
        self.is_malicious = is_malicious  # Whether the node is malicious
        self.preprepare_val = None        # Pre-prepare message (always correct value)
        self.prepare_cnt = defaultdict(int)  # Count of prepare messages received
        self.commit_cnt = defaultdict(int)   # Count of commit messages received
        self.prepared = False               # Whether reach prepared state
        self.committed = False              # Whether reach committed state
        self.result = None                  # Final consensus result
```

PBFT_Simulator Class Attributes:

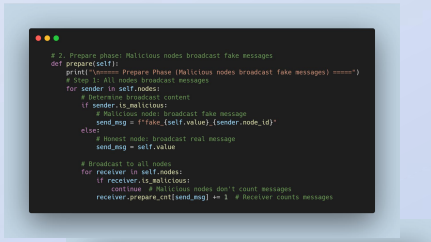
Attribute	Description
N	Total number of consensus nodes
f	Number of malicious nodes in the network
honest_num	Count of honest nodes (total - malicious)
safe	PBFT security check: f ≤ (N-1)//3 (True=valid)
threshold	Consensus approval threshold (≥2/3 nodes)
malicious_ids	Randomly selected malicious node IDs (1~N-1, node 0 is honest primary)
nodes	List storing all Node class instances
honest_node_ids	List of all honest node identifiers
value	Correct proposed value for consensus (block hash)

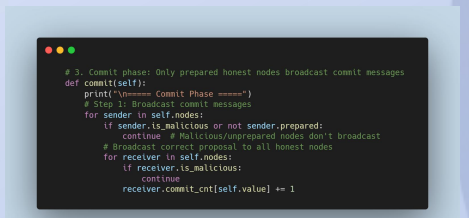
PBFT_Simulator Class Attributes:

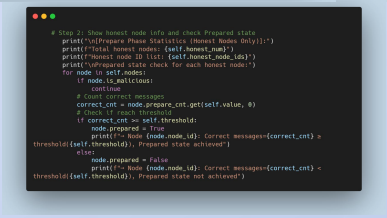
```python
# PBFT Simulator class: Control the entire consensus process
class PBFT_Simulator:
    def __init__(self, total_nodes, malicious_num):
        self.N = total_nodes                # Total number of nodes
        self.f = malicious_num              # Number of malicious nodes
        self.honest_num = total_nodes - malicious_num  # Number of honest nodes

        # Standard PBFT security condition: f ≤ (N-1)/3
        self.safe = (self.f <= (self.N - 1) // 3)

        # Consensus threshold: more than 2/3 of total nodes (rounded up)
        self.threshold = (2 * self.N) // 3 + 1

        # Create nodes: Node 0 is primary node (always honest)
        # Randomly select f nodes from 1~N-1 as malicious nodes
        malicious_ids = random.sample([i for i in range(1, self.N)], self.f) if self.N > 1 else []
        self.nodes = []
        self.honest_node_ids = []  # Store all honest node IDs
        for i in range(self.N):
            is_mal = (i in malicious_ids)
            self.nodes.append(Node(i, is_malicious=is_mal))
            if not is_mal:
                self.honest_node_ids.append(i)

        self.value = "block_hash_123456"  # Correct proposal value
```

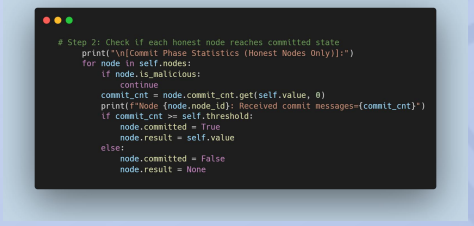
PBFT_Simulator Class Methods:

Method	Description
pre_prepare()	Pre-prepare phase: All nodes receive correct proposal value
prepare()	Prepare phase: Honest nodes broadcast real msg; malicious broadcast fake msg; count & verify prepare state
commit()	Commit phase: Prepared honest nodes broadcast commit msg; count & verify commit state
run()	Execute full PBFT consensus: run pre-prepare → prepare → commit → show result
show_result()	Display final consensus result of honest nodes; judge consensus success









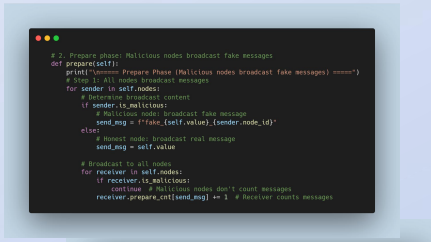
PBFT_Simulator Class Methods:

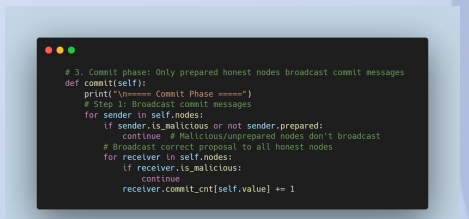
Method	Description
pre_prepare()	Pre-prepare phase: All nodes receive correct proposal value

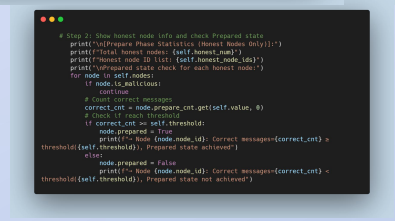
```python
# 1. Pre-prepare phase: All nodes (including malicious) receive correct value
def pre_prepare(self):
    print("\n===== Pre-prepare Phase (All nodes receive correct proposal) =====")
    for node in self.nodes:
        node.preprepare_val = self.value  # Key rule: All nodes get correct value!
        status = "Malicious" if node.is_malicious else "Honest"
        print(f"Node {node.node_id} [{status}] received: {node.preprepare_val}")
```

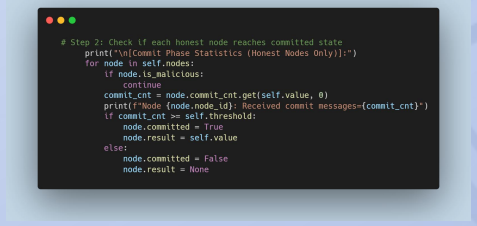
PBFT_Simulator Class Methods:

Method	Description
pre_prepare()	Pre-prepare phase: All nodes receive correct proposal value
prepare()	Prepare phase: Honest nodes broadcast real msg; malicious broadcast fake msg; count & verify prepare state
commit()	Commit phase: Prepared honest nodes broadcast commit msg; count & verify commit state
run()	Execute full PBFT consensus: run pre-prepare → prepare → commit → show result
show_result()	Display final consensus result of honest nodes; judge consensus success









PBFT_Simulator Class Methods:

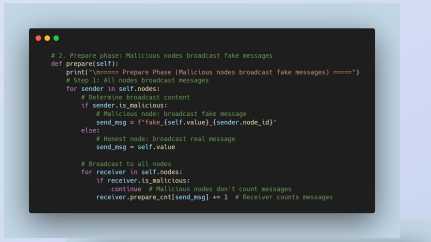
Method	Description
prepare()	Prepare phase: Honest nodes broadcast real msg; malicious broadcast fake msg; count & verify prepare state

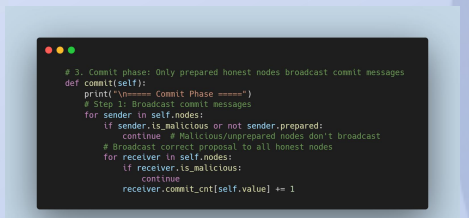
```python
# 2. Prepare phase: Malicious nodes broadcast fake messages
def prepare(self):
    print("\n===== Prepare Phase (Malicious nodes broadcast fake messages) =====")
    # Step 1: All nodes broadcast messages
    for sender in self.nodes:
        # Determine broadcast content
        if sender.is_malicious:
            # Malicious node: broadcast fake message
            send_msg = f"fake_{self.value}_{sender.node_id}"
        else:
            # Honest node: broadcast real message
            send_msg = self.value

        # Broadcast to all nodes
        for receiver in self.nodes:
            if receiver.is_malicious:
                continue  # Malicious nodes don't count messages
            receiver.prepare_cnt[send_msg] += 1  # Receiver counts messages
```

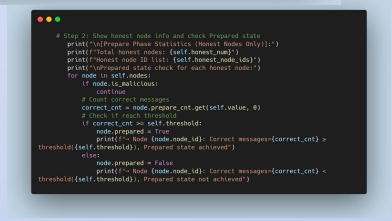
```python
    # Step 2: Show honest node info and check Prepared state
        print("\n[Prepare Phase Statistics (Honest Nodes Only)]:")
        print(f"Total honest nodes: {self.honest_num}")
        print(f"Honest node ID list: {self.honest_node_ids}")
        print("\nPrepared state check for each honest node:")
        for node in self.nodes:
            if node.is_malicious:
                continue
            # Count correct messages
            correct_cnt = node.prepare_cnt.get(self.value, 0)
            # Check if reach threshold
            if correct_cnt >= self.threshold:
                node.prepared = True
                print(f"→ Node {node.node_id}: Correct messages={correct_cnt} ≥
threshold({self.threshold}), Prepared state achieved")
            else:
                node.prepared = False
                print(f"→ Node {node.node_id}: Correct messages={correct_cnt} <
threshold({self.threshold}), Prepared state not achieved")
```

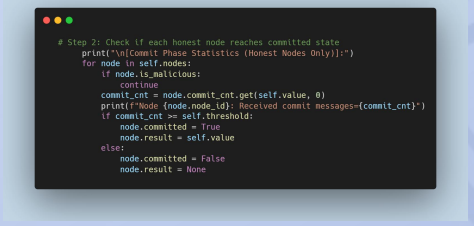
PBFT_Simulator Class Methods:

Method	Description
pre_prepare()	Pre-prepare phase: All nodes receive correct proposal value
prepare()	Prepare phase: Honest nodes broadcast real msg; malicious broadcast fake msg; count & verify prepare state
commit()	Commit phase: Prepared honest nodes broadcast commit msg; count & verify commit state
run()	Execute full PBFT consensus: run pre-prepare → prepare → commit → show result
show_result()	Display final consensus result of honest nodes; judge consensus success









PBFT_Simulator Class Methods:

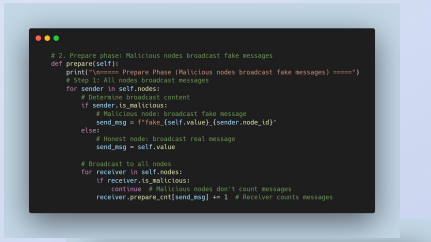
Method	Description
commit()	Commit phase: Prepared honest nodes broadcast commit msg; count & verify commit state

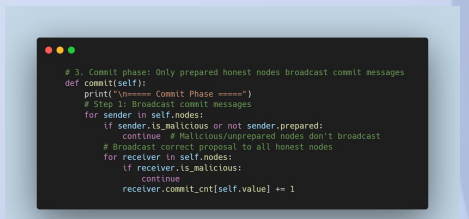
```python
# 3. Commit phase: Only prepared honest nodes broadcast commit messages
def commit(self):
    print("\n===== Commit Phase =====")
    # Step 1: Broadcast commit messages
    for sender in self.nodes:
        if sender.is_malicious or not sender.prepared:
            continue  # Malicious/unprepared nodes don't broadcast
        # Broadcast correct proposal to all honest nodes
        for receiver in self.nodes:
            if receiver.is_malicious:
                continue
            receiver.commit_cnt[self.value] += 1
```

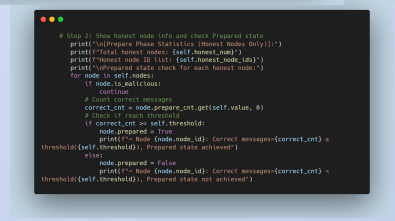
```python
# Step 2: Check if each honest node reaches committed state
    print("\n[Commit Phase Statistics (Honest Nodes Only)]:")
    for node in self.nodes:
        if node.is_malicious:
            continue
        commit_cnt = node.commit_cnt.get(self.value, 0)
        print(f"Node {node.node_id}: Received commit messages={commit_cnt}")
        if commit_cnt >= self.threshold:
            node.committed = True
            node.result = self.value
        else:
            node.committed = False
            node.result = None
```

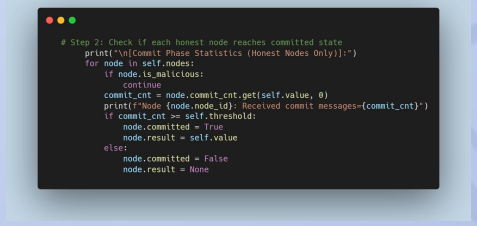
PBFT_Simulator Class Methods:

Method	Description
pre_prepare()	Pre-prepare phase: All nodes receive correct proposal value
prepare()	Prepare phase: Honest nodes broadcast real msg; malicious broadcast fake msg; count & verify prepare state
commit()	Commit phase: Prepared honest nodes broadcast commit msg; count & verify commit state
run()	Execute full PBFT consensus: run pre-prepare → prepare → commit → show result
show_result()	Display final consensus result of honest nodes; judge consensus success









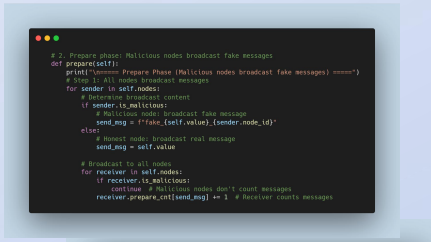
PBFT_Simulator Class Methods:

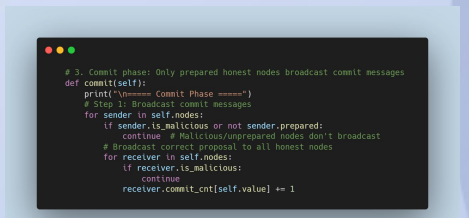
Method	Description
run()	Execute full PBFT consensus: run pre-prepare → prepare → commit → show result

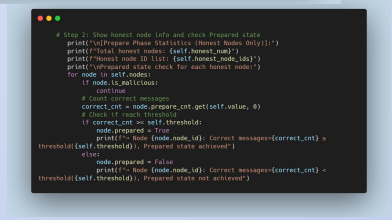
```
# Run the complete PBFT consensus process
    def run(self):
        print("===== PBFT Consensus Simulation Started =====")
        print(f"Total nodes: {self.N} | Malicious nodes: {self.f} | Honest nodes: {self.honest_num}")
        print(f"PBFT Security Threshold: f ≤ (N-1)/3 → Current {self.f} ≤ {(self.N-1)//3} → {self.safe}")
        print(f"Consensus Threshold (≥2/3 of total nodes): {self.threshold}")

        self.pre_prepare()
        self.prepare()
        self.commit()
        self.show_result()
```

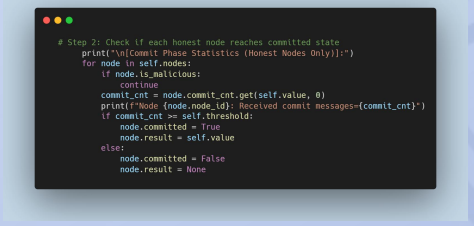
PBFT_Simulator Class Methods:

Method	Description
pre_prepare()	Pre-prepare phase: All nodes receive correct proposal value
prepare()	Prepare phase: Honest nodes broadcast real msg; malicious broadcast fake msg; count & verify prepare state
commit()	Commit phase: Prepared honest nodes broadcast commit msg; count & verify commit state
run()	Execute full PBFT consensus: run pre-prepare → prepare → commit → show result
show_result()	Display final consensus result of honest nodes; judge consensus success









PBFT_Simulator Class Methods:

Method	Description
show_result()	Display final consensus result of honest nodes; judge consensus success

```python
# Show final consensus result
def show_result(self):
    print("\n===== Final Consensus Result ======")
    honest_results = []
    for node in self.nodes:
        if node.is_malicious:
            continue
        print(f"Honest Node {node.node_id} Final Result: {node.result}")
        honest_results.append(node.result)

    # Judge if consensus succeeded
    if len(set(honest_results)) == 1 and honest_results[0] is not None:
        print("\n✅ Consensus Success! All honest nodes reached agreement")
    else:
        print("\n❌ Consensus Failed! Honest nodes did not reach agreement")
```

Additional Methods:

Attribute/Method	Description
get_input()	Open GUI dialog to get PBFT node parameters; validate inputs and return valid total & malicious nodes

```python
# Get user input via GUI dialog
def get_input():
    # Initialize tkinter and hide main window
    root = tk.Tk()
    root.withdraw()

    # Get total node count (must be ≥4)
    while True:
        total_input = simpledialog.askstring("Input Total Nodes", "Please enter total number of PBFT nodes (positive integer ≥4): ", parent=root)
        if total_input is None:  # User cancelled input
            exit()
        # Validate input
        try:
            total_nodes = int(total_input)
            if total_nodes >= 4:
                break
            else:
                messagebox.showerror("Input Error", "Total nodes must be ≥4!")
        except ValueError:
            messagebox.showerror("Input Error", "Please enter a valid positive integer!")
```

```python
    # Get malicious node count (0 ≤ malicious nodes < total nodes)
    while True:
        malicious_input = simpledialog.askstring("Input Malicious Nodes", f"Total nodes={total_nodes}, please enter number of malicious nodes (0 ≤ number < {total_nodes}): ", parent=root)
        if malicious_input is None:  # User cancelled input
            exit()
        # Validate input
        try:
            malicious_count = int(malicious_input)
            if 0 <= malicious_count < total_nodes:
                break
            else:
                messagebox.showerror("Input Error", f"Malicious nodes must satisfy: 0 ≤ number < {total_nodes}!")
        except ValueError:
            messagebox.showerror("Input Error", "Please enter a valid integer!")

    return total_nodes, malicious_count
```

Main Programme:

```python
# Main program entry
if __name__ == "__main__":
    # Get user input
    N, F = get_input()
    # Initialize simulator and run
    sim = PBFT_Simulator(N, F)
    sim.run()
```

Simulation Scenario Design

To verify the core conclusions of PBFT, two comparative scenarios are designed. Both use **9 total nodes (N=9)**.

The security threshold is **f ≤ (9−1)/3 = 2**, meaning consensus succeeds if malicious nodes ≤ 2, and fails otherwise.

Scenario	Total Nodes	Malicious Nodes	Honest Nodes	Meets SecurityThreshold	Expected Result
Scenario 1	9	1	8	Yes (1 ≤ 2)	Consensus success; all honest nodes reach agreement
Scenario 2	9	3	6	No (3 > 2)	Consensus failure; no honest nodes reach final decision

Simulation Actual Results

Consensus Success

Scenario	Total Nodes	Malicious Nodes	Honest Nodes	Meets SecurityThreshold	Expected Result
Scenario 1	9	1	8	Yes (1 ≤ 2)	Consensus success; all honest nodes reach agreement

Pre-prepare phase

The primary node broadcasts a correct proposal. Only 1 malicious node tampers with the proposal, while 8 honest nodes receive the correct one.

Prepare phase

All 8 honest nodes broadcast valid prepare messages. Each honest node receives 8 correct prepare messages, well above the threshold **(2×8)//3 + 1 = 6**, so all honest nodes confirm the proposal is valid.

Commit phase

All 8 honest nodes broadcast commit messages. Each honest node receives 8 correct commit messages, exceeding the threshold. Finally, all honest nodes reach consistent consensus with the original proposal value.

Simulation Actual Results

Consensus Failure

Scenario	Total Nodes	Malicious Nodes	Honest Nodes	Meets SecurityThreshold	Expected Result
Scenario 2	9	3	6	No (3 > 2)	Consensus failure; no honest nodes reach final decision

Pre-prepare phase

Some of the 3 malicious nodes tamper with the proposal. 6 honest nodes receive the correct proposal, but invalid messages from malicious nodes interfere with the statistics in the prepare phase.

Prepare phase

honest nodes broadcast correct prepare messages. Each honest node only receives 5 valid prepare messages, which **does not exceed the threshold ((2×6)//3 + 1 = 5)**, so the proposal cannot be confirmed.

Commit phase

No honest nodes confirm the proposal, so no commit messages are broadcast. The final decision of all honest nodes is None, and consensus cannot be reached.

Core Conclusion Verification

The results of the two scenarios fully align with the core conclusion of the PBFT algorithm

the system can reach effective consensus when the number of malicious nodes ≤ (N−1)/3,and fails to reach consensus when the number of malicious nodes > (N−1)/3.

This project successfully verified the conclusion through pure logical simulation and achieved the expected goals.

4 Core Challenges

Logic Cohesion

Three-phase flow integrity

Ensuring Prepare/Pre-prepare/Commit stages

Threshold Precision

2/3 calculation accuracy

Precise computation for safety thresholds

Malicious Behavior

Realistic attack simulation

Testing robustness under Byzantine faults

Broadcast Logic

Accurate message passing

Avoiding duplication and omission

Challenge 1 —— Logic Cohesion

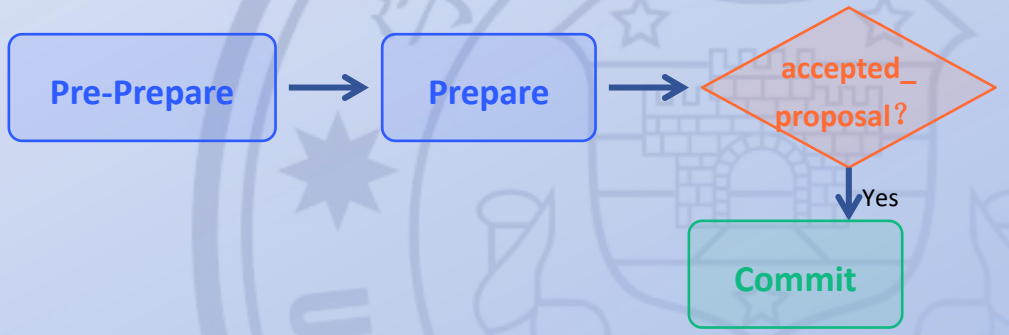
Problem

Nodes may skip the "Prepare" phase and directly jump to "Commit", breaking consensus integrity.

Solution

Introduce an`accepted_proposal`gatekeeper flag to enforce the three-phase sequence.

Workflow Enforcement



Challenge 2 —— Threshold Precision

The Problem

Integer division errors can inadvertently lower the safety threshold, leading to potential system vulnerabilities.

The Solution

Implement ceiling division to ensure the threshold is always rounded up, maintaining the 2/3 safety requirement.

Mathematical Definition

$$\text{threshold} = \lfloor 2N / 3 \rfloor + 1$$

Where N = total number of honest nodes

Validation: Example Calculations

Total Nodes	2/3 Threshold	Our Formula
4	2.67 → 3	$\lfloor 2.67 \rfloor + 1 = 3$
7	4.67 → 5	$\lfloor 4.67 \rfloor + 1 = 5$
10	6.67 → 7	$\lfloor 4.67 \rfloor + 1 = 5$

Result: The formula consistently rounds up to the correct safety threshold.

Challenge 3 —— Malicious Behavior

Problem: Simulate realistic Byzantine faults to test PBFT resilience

Solution: 50% chance to corrupt proposal + no Prepare/Commit votes

Honest Node Behavior

- **Propose:** Generates valid block proposals
- **Prepare:** Votes normally during prepare phase
- **Commit:** Votes normally during commit phase

Malicious Node Behavior

- **Corrupt:** 50% chance to send invalid data
- **Silent:** No response during prepare phase
- **Silent:** No response during commit phase

Objective: Verify consensus algorithm can tolerate up to 1/3 malicious nodes and maintain data integrity.

Challenge 4 —— Broadcast Logic

Problem

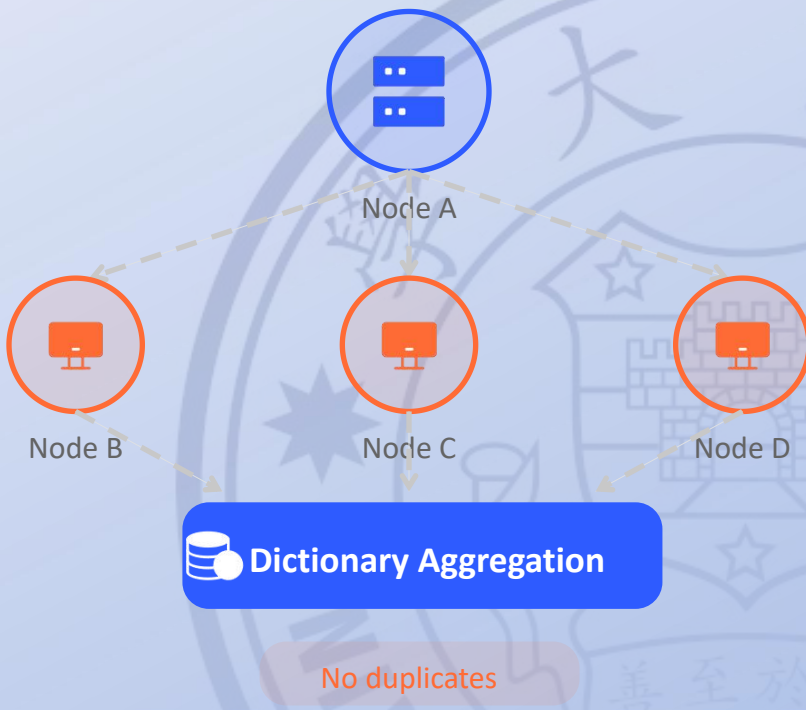
Message duplication or omission in distributed systems.

Solution

Implement **Double-loop** broadcast + **Dictionary** aggregation.

Guarantee

Accuracy and integrity of message delivery with automatic deduplication.



Overview

What We Built

- ✓Pure PBFT simulation
- ✓3-phase consensus process
- ✓Verified 2/3 safety threshold

What We Learned

- Deepened theory understanding
- Python OOP programming skills
- Distributed systems thinking

Where We Go

- 5 key improvement paths
- Enhance production readiness
- Optimization for real-world use

Project Summary — What We Achieved

Core Outcome

Pure Python simulation validating PBFT's 2/3 safety threshold. Successfully implemented all consensus phases without external dependencies.

Three Key Metrics

- Code Lines:~200 (concise)
- Dependencies:0 (pure logic)
- Phases:3/3 fully implemented

Consensus Process Timeline

- Pre-Prepare Phase
- Prepare Phase
- Commit Phase
- Consensus Successfully Reached

Personal Learnings — 3 Dimensions

Theory

PBFT internals
Byzantine math
3-phase workflow

Practice

Python OOP
Debugging logic
Broadcast modeling

Mindset

Distributed thinking
Threshold-first design
Problem decomposition

"Continuous learning is the minimum requirement for success in any field."

Future Work — 5 Directions

Future Work Directions

01. View Change

Impact: Dynamic leader election mechanism

02. Rich Malice

Impact: Simulate equivocation & selective silence

03. Visualization

Impact: Real-time message flow graphs

04. Multi-Proposal

Impact: Concurrent request handling capability

05. Optimization

Impact: Scale system to support 100+ nodes

Development Roadmap

Phase 1: View Change

Implementing dynamic leader election.

Phase 2: Rich Malice

Adding advanced malicious node behaviors.

Phase 3: Visualization

Building real-time message flow graphs.

Phase 4: Multi-Proposal

Enabling concurrent request handling.

Phase 5: Optimization

Scaling to support 100+ nodes.

References & Closing

Key Reference

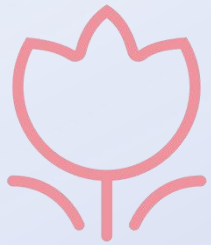
Castro, M., & Liskov, B. (1999).*Practical Byzantine Fault Tolerance*.Proceedings of OSDI.

Additional References

- Blockchain Technology Principles and Applications (2nd Edition). Cai Weide et al.
- Python Object-Oriented Programming. Dong Fuguo.

Closing Statement

"A clean, verifiable simulation that bridges theory and practice — foundation for deeper distributed systems exploration."



Thanks For Listening!